

Design and Analysis of Algorithms

Week 5: Greedy Algorithms I – Classic Paradigm & Problems

TOAE209/TOAH209 – Foreign Trade University

Department of Technology & Data Science

Semester 2024–2025

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching
- 7 Summary

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching
- 7 Summary

What is the Greedy Paradigm?

- At each step, choose the option that looks **best in the short term**.
- Make a locally optimal choice in the hope that it leads to a globally optimal solution.
- Unlike Dynamic Programming, once a choice is made, it is **never reconsidered** (no backtracking).

Greedy Invariants

Usually characterized by:

- 1 **Greedy choice property:** A global optimum can be reached by making local choices.
- 2 **Optimal substructure:** An optimal solution contains optimal solutions to subproblems.

Greedy vs Dynamic Programming

Dimension	Greedy Paradigm	Dynamic Programming
Choice	Local choice at each step	Explores all subproblems
Backtracking	Never backs up	Evaluates all overlapping states
Proof of Correctness	Always required (often hard)	Checked via recursive induction
Efficiency	Usually $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$	$\mathcal{O}(n^2)$ or higher

Warning

Greedy algorithms are easy to design, but ****very hard to prove correct****. Most intuitive greedy heuristics are actually incorrect!

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching
- 7 Summary

Coin Changing Problem

Problem Statement

Given an amount A and a set of coin denominations $C = \{c_1, c_2, \dots, c_d\}$, select the minimum number of coins that sum exactly to A .

- ****Greedy Heuristic:**** Choose the largest denomination coin $c \leq A$, add it to the solution, and repeat for $A - c$.
- ****Example (US System: $\{25, 10, 5, 1\}$):**** For $A = 41$:
 - Select 25 (remaining: 16)
 - Select 10 (remaining: 6)
 - Select 5 (remaining: 1)
 - Select 1 (remaining: 0)
 - Total coins: 4 ($25 + 10 + 5 + 1$) – ****Optimal!****

Coin Changing: The Limits of Greedy

Does the greedy heuristic work for *every* coin system?

No! Counter-example

Suppose $C = \{4, 3, 1\}$ and we want to change $A = 6$.

- ****Greedy Selection:****
 - Select 4 (remaining: 2)
 - Select 1 (remaining: 1)
 - Select 1 (remaining: 0)
 - Total: 3 coins ($4 + 1 + 1$)
- ****Optimal Selection:****
 - Select 3 and 3
 - Total: 2 coins ($3 + 3$)

Greedy is ****sub-optimal**** here! General coin changing requires Dynamic Programming.

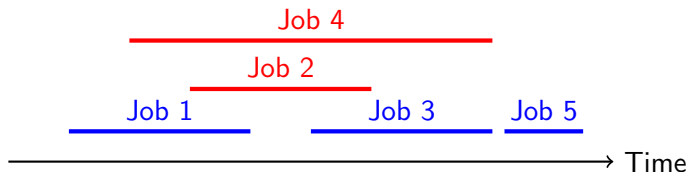
Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling**
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching
- 7 Summary

Interval Scheduling Problem

Problem Statement

Given a set of intervals $J = \{1, 2, \dots, n\}$ where each interval i has a start time s_i and a finish time f_i . Find the ****maximum size subset**** of mutually compatible (non-overlapping) intervals.



Interval Scheduling: Heuristics

Which greedy sorting strategy is optimal?

- ****Strategy 1: Earliest Start Time First**** – Counterexample: one long interval covering the whole timeline prevents all others.
- ****Strategy 2: Shortest Interval First**** – Counterexample: one short interval overlapping with two long compatible ones.
- ****Strategy 3: Fewest Conflicts First**** – Counterexample: complicated overlapping clusters.
- ****Strategy 4: Earliest Finish Time First (EFT)**** – ****Optimal!****

EFT Rule

Sort by finish time f_i ascending. Select the first interval, then select the next compatible interval, and repeat.

Interval Scheduling Pseudocode

```
def interval_scheduling(intervals):  
    # Sort intervals by finish time  
    sorted_ints = sorted(enumerate(intervals), key=lambda x: x  
                          [1][1])  
    selected = []  
    last_end = -float('inf')  
    for idx, (start, end) in sorted_ints:  
        if start >= last_end:  
            selected.append(idx)  
            last_end = end  
    return selected
```

Complexity

* **Time:** $\mathcal{O}(n \log n)$ due to sorting. * **Space:** $\mathcal{O}(n)$ to store indices and sorted list.

Proof of Optimality: Greedy Stays Ahead

Let $A = \{i_1, i_2, \dots, i_k\}$ be the greedy selection, and $O = \{j_1, j_2, \dots, j_m\}$ be an optimal selection.

Theorem

Greedy is optimal, i.e., $k = m$.

Proof.

We prove by induction that for all $r \leq k$:

$$f(i_r) \leq f(j_r)$$

* **Base case ($r = 1$):** Greedy picks the interval with earliest finish time overall, so $f(i_1) \leq f(j_1)$ holds. * **Inductive step:** Assume $f(i_{r-1}) \leq f(j_{r-1})$. Since O is compatible, $s(j_r) \geq f(j_{r-1}) \geq f(i_{r-1})$. Thus, j_r is in the set of candidate intervals available to greedy. Since greedy selects the available interval with earliest finish time, $f(i_r) \leq f(j_r)$. By induction, this holds. If $m > k$, optimal could select j_{k+1} , but $s(j_{k+1}) \geq f(j_k) \geq f(i_k)$, meaning greedy would have picked at least one more interval. Contradiction.

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning**
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching
- 7 Summary

Interval Partitioning Problem

Problem Statement

Given n intervals representing lectures, schedule all of them in the **minimum number of classrooms** such that no two overlapping lectures share a classroom.

Classroom

Class 1

Class 3

Classroom B

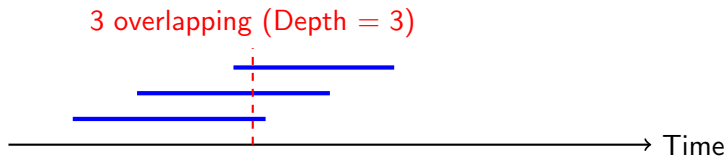
Class 2

Class 4

Interval Partitioning: Lower Bound via Depth

Definition

Depth The **depth** of a set of intervals is the maximum number of intervals that overlap at any single point in time.



Lower Bound

Since all intervals overlapping at time t must be assigned to different rooms, the number of classrooms needed is **at least** the depth d .

Interval Partitioning: Greedy Strategy

Greedy Algorithm

1. Sort intervals by **start time** s_i ascending. 2. Go through each interval: * If an existing classroom is free, assign the interval to it. * If all active classrooms are busy, open a **new classroom**.

Implementation Hint

Store classrooms in a **Min-Heap**, keyed by the finish time of the last lecture assigned to them. * Extract min tells you if a classroom is free. * Heap size at the end represents the number of rooms needed. * **Time Complexity:** $\mathcal{O}(n \log n)$, **Space Complexity:** $\mathcal{O}(n)$.

Interval Partitioning: Proof of Optimality

Proof.

Let d be the maximum depth of the interval set. We show that greedy uses exactly d classrooms. * Suppose greedy allocates classroom k . This occurs because when scheduling some interval i , all $k - 1$ active classrooms are busy. * This means at the start time s_i , there are $k - 1$ other intervals currently running. * Including interval i , there are k intervals overlapping at time s_i . * Thus, the depth at s_i is at least k . * Since max depth is d , we must have $k \leq d$. * Therefore, the number of classrooms allocated by greedy is at most d . * Since d is a mathematical lower bound, greedy uses **exactly** d classrooms, which is optimal! $\square$

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness**
- 6 Optimal Offline Caching
- 7 Summary

Scheduling to Minimize Lateness

Problem Statement

We have a single machine and n jobs. Each job i has a processing time t_i and a deadline d_i . If we start at time 0, and job i finishes at time f_i , its lateness is:

$$L_i = \max(0, f_i - d_i)$$

Find a schedule to **minimize the maximum lateness** $L = \max_i L_i$.

- **Heuristic 1:** Shortest processing time first – Counterexample: ignores deadlines.
- **Heuristic 2:** Smallest slack time first ($d_i - t_i$) – Counterexample: breaks down easily.
- **Heuristic 3:** Earliest Deadline First (EDF) – Sort by deadline d_i ascending.
Optimal!

Minimize Lateness: Proof of Optimality

We prove EDF is optimal using an **exchange argument**.

Definition

Inversion An **inversion** in a schedule is a pair of jobs i and j such that job i is scheduled before job j , but job j has an earlier deadline ($d_j < d_i$).

- An EDF schedule has **zero inversions** and no idle time.
- **Lemma:** Swapping two adjacent inverted jobs does not increase the maximum lateness.
- **Proof Sketch:** Let i and j be adjacent inverted jobs. Swapping them only changes their individual finish times. Job j finishes earlier (reducing its lateness). Job i finishes at j 's old finish time, which is less than j 's deadline. The lateness of the swapped pair is no worse than before.
- By repeatedly swapping adjacent inversions, we can transform any optimal schedule into the EDF schedule without increasing maximum lateness.

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching**
- 7 Summary

Optimal Offline Caching Problem

Problem Statement

Given a cache of capacity k and a sequence of memory page requests $r_1, r_2, \dots, r_N$. When a requested page is not in the cache (a **cache miss**), we must bring it in. If the cache is full, we must evict one page. Minimize the total number of cache misses.

- **Online Caching:** The future requests are unknown (e.g., LRU, FIFO).
- **Offline Caching:** The complete future request sequence is known.
- **Optimal Strategy (Belady):** Furthest-in-Future (FIF)

FIF Rule

When a cache miss occurs and the cache is full, evict the page that is requested **furthest in the future**.

Caching: FIFO Optimality and LRU Comparison

- FIFO is mathematically optimal (minimizes cache misses for any request stream).
- Proof utilizes a complex exchange argument, showing any optimal schedule can be aligned to FIFO step-by-step.

Benchmarking Replacement Policies

On patterned requests of size 100 with a cache size $k = 3$, look at typical cache miss results:

- ****FIFO (First-in First-out):**** 26 misses
- ****LRU (Least Recently Used):**** 30 misses
- ****FIF (Optimal Furthest-in-Future):**** ****21 misses**** (Strictly optimal!)

Note

Real browsers and CPUs cannot use FIF because they cannot predict the future. However, they use LRU as an online approximation of Furthest-in-Future!

Outline

- 1 The Greedy Paradigm
- 2 Coin Changing
- 3 Interval Scheduling
- 4 Interval Partitioning
- 5 Scheduling to Minimize Lateness
- 6 Optimal Offline Caching
- 7 Summary**

Summary of Classic Greedy Algorithms

- 1 ****Coin Changing:**** Optimal for canonical systems, sub-optimal in general.
- 2 ****Interval Scheduling:**** Sort by finish time (EFT). Proof: Greedy stays ahead.
- 3 ****Interval Partitioning:**** Sort by start time. Proof: Number of rooms = Max Depth.
- 4 ****Minimizing Lateness:**** Sort by deadline (EDF). Proof: Exchange argument.
- 5 ****Optimal Caching:**** Evict furthest-in-future (FIF). Proof: Exchange argument.

Next Steps

* Complete programming problems in `starter.py`. * Run `demo.py` to see the visualizations of all 5 algorithms. * Attempt proof problems in `exercises.md`.